Learning from the User Without Learning Their Mistakes:
A Preregistered Study of On-Device Personalisation for a Small Language Model

Erik Thye

Abstract

Small language models that run entirely on a user's computer can adapt to that user without sending text elsewhere, but a model that learns from what its user writes also learns the user's typos and grammar mistakes. We study this with Sela-2, a decoder-only transformer with 352 M parameters per token, trained from scratch on one rented GPU for about USD 23, that runs in int8 on a CPU inside an offline desktop assistant. The assistant learns in three layers: a note of the user's facts in the system prompt, the conversation thread, and a k-nearest-neighbour language memory over the user's messages. An error-aware admission filter guards the memory: the model itself judges, on the whole message, whether a word looks like a mistake, and a sentence with a clear mistake is not learned at all. Every version was preregistered and measured once on 40 users it had never seen; we report all six registered measurements, including a failed first version, a flawed decision rule and a counting error. The fact note raised the use of a user's facts from 0 % to 20–53 % of the replies that called for them; from the second version on, at most two replies per run repeated an injected mistake. The memory's gain on held-out messages fell from 3.39 % on development users to 1.45, 0.78 and 0.09 % on three fresh synthetic groups. A 60-setting grid did not recover it; the gain tracked how many word pairs of a held-out sentence the user had already written (Spearman $\rho = 0.42$, 95 % CI [0.24, 0.57]), a property we had set ourselves when writing the synthetic sentences. On real WildChat-1M users, the memory reduced bits per byte by 1.59 % (95 % CI [0.50, 2.91]) in the deciding test and by 3.07 % in a second group. A relevance filter with cue words frozen before blind-written probes existed cut unrequested mentions of personal facts from 24.7 % to 0.0 % of probe replies without reducing fact use.

1 Introduction

A personal assistant becomes more useful when it adapts to its user: it should remember that the user's dog is called Rex, and it may come to write a little more like the user does. For privacy, the most attractive place for such adaptation is the user's own computer. A small model that runs locally can learn from a user's messages without sending them anywhere, and whatever it has learned can be deleted exactly. But users make mistakes. A model that learns how its user writes also learns the user's misspellings (*definately*), merged words (*alot*; German *garnicht* for *gar nicht*) and grammar slips (*she go*), and may repeat them back to the person who made them. The first version of the system studied here did exactly that: its language memory put *alot* and *garnicht* into 6 of 280 replies (§ 6.1).

This paper is about learning from a user without learning their mistakes, and about evaluating such a system honestly. The second part is as hard as the first. Personalisation is often developed and tested on synthetic personas or a few users; each iteration checked on the same users risks fitting them, and with small samples, flexible analysis can support almost any conclusion [17, 53]. We therefore ran the work as a sequence of preregistered measurements [40, 41]: before each version was measured, its measurement, criteria and decision rule were committed to the project repository; the version was then measured once on 40 users it had never seen, and the rule decided whether it shipped.

The substrate is Sela-2, a decoder-only transformer with 352 M parameters per token, trained from scratch with its own English–German tokenizer on one rented H100 GPU for about USD 23 (§ 3.1). It runs in int8 on a CPU inside ENGRAMM, an offline desktop assistant, and learns in three layers (Fig. 1): (A) a note of the user's facts in the system prompt, (B) the conversation thread, and (C) a k-nearest-neighbour (kNN) language memory [29] over the user's own messages, guarded by an error-aware admission filter. Our contributions:

- **On-device personalisation for a small model trained from scratch** (§ 3): three learning layers whose every learned item stays on the user's machine and can be forgotten exactly, at a writing-speed cost of 2–4 %.
- **An error-aware admission filter for kNN language memories** (§ 4): the model judges whole messages against small variants and keeps sentences with clear mistakes out. We quantify its trade-off between stored mistakes and discarded correct text and chose its letter rule by a criterion fixed before the result.
- **A preregistered, sequential evaluation** (§ 5): six registered measurements on 240 fresh users (160 synthetic, 80 real) with fixed criteria and rules. We report every outcome, including a failed first version, a rule that coupled unrelated criteria, a development detour, and a counting error.
- **A lesson about synthetic test users** (§ 6.3): the memory's gain fell from +3.39 % to +0.09 % across synthetic groups; the evidence points not to its settings but to the held-out sentences, which we wrote and which reused less and less of the users' wording. On real WildChat-1M users

[64] it met the registered criterion (+1.59 %, 95 % CI [0.50, 2.91]).

- **Fact intrusion and a relevance gate** (§ 6.5): with all facts in its note, the small model volunteered unrequested personal facts in a quarter to two fifths of its probe replies; a relevance filter with frozen cue words removed this on blind-written probes without lowering fact use.

Throughout, the scale is small: Sela-2 is a weak chat model by current standards, each group has 40 users, and the effects are modest.

2 Related work

Cache and nearest-neighbour language models. Cache models raised the probability of recently used words [34]; continuous caches retrieve past hidden states by similarity [18]; dynamic evaluation adapts the weights while reading [33]. The kNN-LM [29] interpolates the next-token distribution with one formed from the nearest stored contexts, and the same datastore idea adapts translation models to new domains [30]. kNN-LM gains depend strongly on n-grams shared by datastore and evaluation text [14]; Xu et al. [59] analyse why the method works, and lower perplexity need not mean better open-ended generation [56]. Retrieval-augmented models condition on retrieved text instead [7, 35]. Layer C is a small per-user kNN-LM; what is new is the control over what may enter its datastore, and our lexical-reuse analysis is a per-user counterpart of the overlap finding of Drozdov et al. [14].

Personalisation and memory. Persona-conditioned dialogue [63], personalised generation from user profiles [51] and long-term memory for LLM agents [42, 43, 65] place relevant user information in the prompt; layer A is a minimal version for a small model. Earlier work personalised language models to an individual's text by interpolation and priming [31], and on-device training keeps keyboard data on phones [20]. Recent benchmarks study over-personalisation, where memory-augmented assistants force irrelevant personal information into replies [16, 22]; we document the same failure in a much smaller model and test a simple remedy. We know of no work on keeping a user's own mistakes out of what a personalised model learns.

Spelling and grammatical errors. Noisy-channel spelling correction ranks corrections with a language model and an error model [28]; language-model-based grammatical error correction compares the likelihood of variants [10]; see Bryant et al. [11]. Our filter borrows the comparison of variants by likelihood but corrects nothing: it only decides what not to learn.

Evaluation practice. Preregistration separates confirmatory from exploratory analysis [40]; undisclosed analytic flexibility inflates false positives [17, 53]; machine learning has piloted preregistered papers [4] and reproducibility checklists [45]. Data made by people carry their makers' choices: annotation artifacts let models solve inference tasks from the hypothesis alone [19], and synthetic noise differs from natural noise [2]. Our lexical-reuse finding is an instance of this problem in personalisation.

Privacy and forgetting. Language models can memorise and leak training data [12], and privacy for text is hard to define [9]. Removing data from weights needs dedicated unlearning [8]; a non-parametric memory forgets exactly by deleting entries.

3 System

3.1 Sela-2, a small model trained from scratch

Sela-2 follows the nanochat recipe [27], a descendant of nanoGPT [26] and the modded-nanogpt speedruns [25]: 24 pre-norm decoder layers of width 1,024 with 8 heads, a context of 2,048 tokens, rotary embeddings [55], query–key normalisation [21], RMSNorm without weights [62], squared-ReLU feed-forward blocks [54], two value embedding tables, an untied output matrix and logits soft-capped at 15. Of its 503,316,938 stored parameters, 352,321,920 are used per token (value embeddings are looked up, not multiplied). Its byte-level BPE vocabulary [52] of 49,152 tokens averages 4.67 bytes per token on English and 4.39 on German.

Pretraining consumed 3.06 B tokens on one H100 (4.93 h, about 172 k tokens/s) from FineWeb-Edu [44], DCLM [36], C4 [48], Cosmopedia, Wikipedia, PG-19 [47] and German FineWeb2-HQ [38]; the final 30 % of training added chat, grounded question-answering and identity conversations (SmolTalk [3] and SmolTalk2, OpenAssistant [32], Dolly, SQuAD 2.0 [49], HotpotQA [60], GermanRAG). Muon [24] with Polar Express [1] and NorMuon [37] updated the block matrices, AdamW the rest; 657 steps of supervised fine-tuning on 78.7 M assistant tokens followed. The rented session lasted about 5.7 h and cost about USD 23, including a failed first start, under a USD 25 cost limit fixed before the run.

On a pooled zero-shot score over six tasks (HellaSwag, PIQA, ARC-Easy/Challenge, WinoGrande, OpenBookQA; 6, 13, 39, 50, 61), measured with one harness for all models, the base model scores 45.5 against 38.9 for GPT-2 small [46] (paired bootstrap 95 % CI of the difference 5.7–7.6), 42.9 for GPT-2 medium, and 39.5 and 43.5 for Pythia-160M and Pythia-410M [5]; a token-level contamination audit leaves the pooled score unchanged. The chat fine-tuning cost about four pooled points, so by a rule fixed before that comparison the app ships the interpolation $0.7 \cdot \text{base} + 0.3 \cdot \text{chat}$ ("mix30", cf. WiSE-FT [58]; pooled 45.7). Quantising the chat model to int8 cost 0.001 bits per byte. Sela-2 is nevertheless a weak chat model: its free replies are fluent but often factually wrong, and it missed several of its own preregistered chat targets (on IFEval [66] it gained at most 4.8 points over the base model, against a target of 8). We study the personalisation of a small, imperfect model, not of a capable assistant.

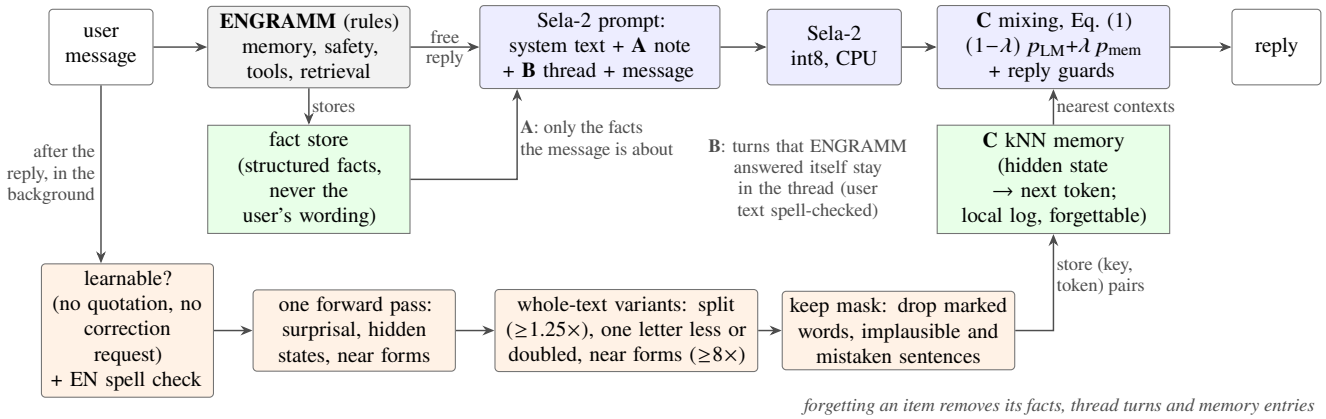


Figure 1: The three learning layers. ENGRAMM, a rule-based assistant, handles every turn first and stores the user's facts. **A**: a note built from structured facts in Sela-2's system prompt. **B**: turns that ENGRAMM answered itself stay in Sela-2's conversation. **C**: after each reply, a learnable message is analysed by the model itself (§ 4) and its kept tokens are stored as (hidden state, next token) pairs; while Sela-2 writes, the nearest stored contexts are mixed into its next-token distribution.

3.2 The host assistant

Sela-2 runs inside ENGRAMM, an offline desktop assistant that itself uses no neural network: it answers factual questions from a local Wikipedia/Wikidata pack, keeps a rule-based store of what the user tells it, handles safety, tools and clarifying questions, and forgets on request. ENGRAMM handles each turn first. Its memory, safety and tool replies are shown as they are; answers with sources are rewritten by Sela-2 and shown only if every number and name in the rewrite occurs in a cited source; small talk, writing requests and open questions receive Sela-2's free reply, which is what the learning layers affect. The int8 model runs on NumPy with numba kernels; in the measurements from §8 on it wrote 29.8–41.6 tokens per second (F arms) on GitHub-hosted CI runners.

3.3 Three learning layers

A: the fact note. ENGRAMM's fact store holds structured facts extracted from statements ("My dog is called Rex" becomes an owned pet *dog* named *Rex*). Layer A renders up to eight of them into a note at the end of Sela-2's system prompt. The note is built from the structured facts, never from the user's wording, so a typo in the original message cannot enter it; facts about other people, about the past and negated facts are not shown, and the newest value of a single-valued fact wins. Since §10 of the preregistration the note is written in the third person ("About the user (… facts about the user, not about you; use them only when they help): The user's name is Sam. The user lives in Leeds. The user's dog is called Rex."); since §12 it holds only the facts the current message is about (§ 6.5). Two guards act on every free reply: sentences in which Sela-2 states a user fact as its own ("I live in Leeds.") are removed, and so are sentences that repeat the note.

B: the thread. Turns that ENGRAMM answers itself, such as a statement it stores, stay in Sela-2's conversation history; English user text first passes through ENGRAMM's spell checker. Before this layer existed, Sela-2 recommended New York to a user who had just said they were planning a trip to Italy.

C: the language memory. For every token t_j of a learned message, the memory stores a key and the value t_j. The key is Sela-2's normalised final hidden state at the position that predicts t_j, projected to 128 dimensions by a fixed Gaussian random matrix [23] and scaled to unit length. While Sela-2 writes, the current hidden state is projected likewise, the $K = 16$ stored keys with the highest cosine similarities s_i are retrieved, and

$$p(w) = (1-\lambda)\, p_{\mathrm{LM}}(w) + \lambda\, p_{\mathrm{mem}}(w), \qquad (1)$$
$$p_{\mathrm{mem}}(w) \propto \textstyle\sum_{i:\, t_i = w,\ s_i \ge \theta} e^{(s_i - s_{\max})/\tau},$$
$$\lambda = \lambda_{\max}\, \min\!\big(1, \max\!\big(0, \tfrac{s_{\max} - \theta}{1-\theta}\big)\big),$$

with $\lambda_{\max} = 0.10$, similarity floor $\theta = 0.90$ and $\tau = 0.01$, chosen on development users. The memory thus has no influence unless a stored context is very close, and never more than a tenth of the probability. Proposals that the admission filter would judge to be misspellings are removed from p_{mem} before mixing. Messages are learned in the background after the reply, holding the model lock only for single forward passes; the memory keeps at most 20,000 entries (about 1,000 messages), oldest out first.

Forgetting. Thread turns and memory entries that stem from a message ENGRAMM stored carry the identifier of that memory item and are removed with it. Emptying the memory, the reset on the app's learning page, or "forget everything you learned" clears the whole language memory, an append-only text log plus a derived cache of keys.

Algorithm 1 Admitting one user message to the memory (`engramm/lm/sela_learn.py`).

if m has < 4 words or > 800 characters, asks to correct, rewrite or translate, or quotes text **then**
    return (nothing is learned)
end if
if m is English, fix spelling slips with ENGRAMM's spell checker
one pass of Sela-2 over m as a user turn: surprisal n_j and hidden state of each token t_j
near forms: t_j is *near* if one of the 20 likeliest tokens at j is $\geq 8\times$ likelier than t_j and nearly the same string (edit distance ≤ 2, or same first three letters and lengths within 3)
for up to 8 words w (≥ 4 letters), most surprising first **do**
    split variants: splits of w into two single tokens (*alot* → *a lot*)
    letter variants (3 most surprising words of ≥ 5 letters, ≥ 6 nats): one letter removed or one consonant doubled
    score a variant only if the model prefers it at its first changed token, or (letter) it needs fewer tokens
    mark w if a variant makes the whole message $\geq 1.25\times$ (split) or $\geq 8\times$ (letter) likelier, unless it only drops the first letter or a non-doubled last letter (*shape rule*)
end for
confirm up to 6 near tokens with $n_j \geq 6$ nats: the near form must make the whole message $\geq 8\times$ likelier
keep mask: drop words with a near token or a mark; drop sentences with median surprisal > 6.5 nats; drop every sentence with a marked word or a confirmed near form
return (key, token) pairs of the kept tokens

4 Error-aware admission to the memory

The memory learns what follows what in the user's messages, so a misspelt word would be stored as a likely continuation and boosted whenever a similar context recurred. The filter (Alg. 1) keeps mistakes out with the model's own knowledge of the language. Its central operation compares the likelihood of the *whole* message, including the end of the turn, with and without a small change. Token-level comparisons miss many real mistakes because a mistake changes the tokenisation: *alot* must be compared with the two tokens *a lot*, and *wiederrum* with the single token *wiederum*. Scoring a variant costs only the suffix from its first changed token: the key/value cache is rewound to that position, and variants are scored from the last changed position to the first, so earlier text is never recomputed (rewound scores matched full passes to within 0.001 nats). The analysis runs in the background; in the development probe of the final code, which scores more variants than the app, it took 2.6 s per message on average (at most 15.3 s) on a CI runner.

Why drop whole sentences. A sentence built around a mistake teaches its structure even without the wrong word: from *Gestern ich habe gegangen zum Bäcker* the memory would still learn that *ich* follows *Gestern*. A clear mistake therefore removes its sentence. A near form that is merely likelier at its own position is too weak a signal for this (in a development probe it would have discarded 246 of 320 correct sentences; §8 of the preregistration), so it removes its sentence only when the whole message confirms it.

What the filter cannot see. Grammar mistakes made of correct words (*ich habe gegangen*, *wegen dem Wetter*) are caught only when a near form of one token makes the whole message much likelier, and a missing letter is not tried. The filter also cannot know whether an unusual word is a mistake or the user's choice; every check trades stored mistakes against discarded correct text (§ 6.4).

Choosing the letter rule in advance. On new users the letter check also flagged correct words: British *favourite* (the model prefers *favorite*), inflections (*parts* → *part*), German declension. A first counter-measure, requiring the variant to be likelier also as a word on its own, let real mistakes through (*Packet* is a valid English word). We then replayed 18 candidate rules from the recorded traces of one development probe: no outside check, the word alone, or the word in a fixed sentence ("I wrote the word __ down."), at 0, 1, 1.5 or 2.08 nats, each with and without the shape rule. The selection criterion was fixed before the results were inspected: admissible are rules that, over four user sets (real development users and the consumed synthetic users of §7, §9, §10), store no more injected mistakes than the previous rule without outside check; among them, the highest memory gain on the real development users wins.

5 Evaluation protocol

Preregistration. Each version was registered in a section (§7–§12) of `docs/PREREG_SELA2.md`, committed before the commit that triggered its measurement; the measurement ran on GitHub Actions on that commit's code, and the workflow committed the results (App. F). Apart from whitespace, each section's registered text is identical in its triggering commit and the final document; results were appended as subsections. Development used only consumed users (those of earlier measurements) and real development users; the development probe refuses the synthetic §8 test file and all real test files. Before the first measurement, four rounds of code review by separate AI agent instances that had not written the code (five reviewers in the first), with each finding checked by a second such reviewer, found 85 defects, all fixed before anything was measured. The sequence from the registration of §7 to the decision of §12 took place on 19 October 2026 (00:28–18:59 UTC).

Synthetic users. Each synthetic group (§7–§10) has 20 English and 20 German users generated by a fixed script with a new seed and new names, cities, pets, dishes and hobbies. A user writes eight teaching messages: four fact statements (name, city, pet and its name, favourite dish) and four messages about a hobby; three of the latter are followed by a sentence with a typical mistake from a fixed list of 12 English and 12 German mistakes: misspellings (*freind*, *definately*, *Standart*, *Packet*), merged words (*alot*, *garnicht*) and grammar (*she go*, *he don't*, *ich habe gegangen*). Two further hobby sentences are held out (never taught, free of mistakes). Each

Table 1: Criteria; thresholds compare an arm with *off* on the same users and seeds.

	Definition	Threshold
L1	*Fact use*: fact probes whose reply contains the fact (or a listed inflection), on probes Sela-2 answered in all arms (valid only if ≥ 50 % of them)	F−off ≥ +15
L2	*Copied mistakes*: replies containing one of the user's injected wrong forms	≤ off+2
L2b	*Role confusion*: replies stating a user fact as Sela-2's own (fixed patterns)	≤ off+2
L3	*Memory gain*: relative reduction in bits per byte on held-out messages, memory vs. none	≥ 1 %
L4	*App problems*: app prompts with a detected problem (wrong language, identity error, echo, cut-off, role leak, loop, empty, crash)	≤ off+2
L5	*Writing speed* after reading the prompt, FK/F	≥ 0.85
L6	*Intrusion* (§12): probe replies naming a fact the probe did not ask for; app replies naming the taught user's facts	≤ 10 %; ≤ 3

group is built from 10 hobby scripts (five per language), each shared by four users who differ only in facts and mistakes, so a group has 20 distinct held-out sentences, not 80.

Real users. §11 and §12 use real people's messages to a chatbot from WildChat-1M [64] at a fixed revision (App. E). A user is one hashed IP address within one language (English or German), restricted to conversations not flagged as toxic or redacted. Messages count if they read as plain prose (20–300 characters, no code, links or markup), in time order and without near copies; users with six such messages qualify, the first four are taught and the next two held out. Development and test users are separated by a hash of the hashed IP; 20 users per language and split were drawn with a fixed seed, and the selection (identifiers and hashes only) was committed before anything was measured on them. Facts, injected mistakes and probes are added as for synthetic users. Whether the app would learn a message was not used for selection; in the development set 143/160 messages were learnable, most others contain quotations.

Arms, probes and criteria. After teaching, each probe is asked in a new conversation in three arms with the same seed: **off** (no learning; the app before §7), **F** (layers A and B) and **FK** (A, B and C). Each arm starts from the state after teaching; probes are not learned. A user has four *fact probes* whose natural answer uses one fact ("What should I cook tonight? One idea, please."), two *error probes* inviting the correct form of two of their mistakes ("Write one sentence about learning a lot in a short time.") and one hobby probe; §12 adds two neutral requests and uses blind-written fact probes. The 100 fixed prompts of Sela-2's chat evaluation (60 English, 40 German) are also run through the app in each arm after teaching the first test user. Table 1 defines the criteria. From §7 on, A+B were to be switched on if L1, L2, L2b and L4 held for F, and C additionally if L2, L2b and L4 held for FK and L3 and L5 held; a decision required all 40 users, all app arms, no crashed reply and an L1 coverage of at least 50 %. Later sections restricted the rule to the component being decided (App. B).

Uncertainty. Registered decisions compare point estimates with thresholds, and we report them as registered. We add 95 % intervals that were not part of any registration: Wilson intervals [57] for proportions (treating replies as independent; for L1, a bootstrap over users moves the interval ends by at most 2.2 points), and for L3 a percentile bootstrap [15] with 10,000 replicates over the independent units, users for real groups and hobby scripts for synthetic groups. With 10 clusters the synthetic intervals are approximate.

6 Results

6.1 The sequence of measurements

Table 2 lists every registered measurement. The **first version** (§7) showed the facts as an earlier first-person user turn ("My name is Sam. My dog is called Rex.") and used an older filter with stronger mixing ($\lambda_{\max}$=0.30, θ=0.80, τ=0.05). Fact use rose from 0 to 52.5 %, but Sela-2 repeated the fact lines as its own in 7 replies per arm (*Ich heiße Lea.*), the memory copied *alot* and *garnicht* into 6 replies, gained only 0.55 % and slowed generation to 0.79 of the off arm. By the rule, all layers stayed off. The **second version** (§8) moved the facts into a labelled note with a role-confusion guard, introduced the filter of § 4, and took the mixing setting from a grid on the consumed §7 users. On 40 fresh users it met every criterion, and all layers were switched on.

§9 compared two note forms on new users: third-person sentences raised fact use to 44.4 % against 20.0 % for labels (of 160 fact probes; German 40.0 % against 13.8 % of 80). But the registered rule adopted the sentence form only if that run *also* met the criteria of layer C, and C missed L3 (0.78 %). The sentence form was not adopted (†), and C was switched off again. The rule was badly built: it tied a decision about the note to a criterion of the memory, which never sees the note (L3 is identical in both arms). We did not override it; §10 repeated the comparison on 40 further users with a rule about the note alone. Sentences won again (+34.4 against +23.8 points) and became the default; the memory gained 0.09 % and stayed off. §11 measured the memory on real users and switched it on; §12 tested the relevance filter.

Facts. Moving the facts from a first-person turn into a note removed role confusion (L2b: 7 replies per arm in §7, 0 in every later run). Writing the note as sentences instead of labels raised fact use on two fresh groups by 24.4 and 10.6 points; the change was motivated by development runs in which, with labels (*Name: Lea; Hund: Bruno*), the small model often took the user's name for the pet's. Fact use was lower in German than in English in most runs (27.5 % against 41.2 % of 80 each with sentences in §10). L1 counts any mention of a fact, fitting or not; that weakness led to §12.

6.2 Where the mistakes went

Table 3 follows each injected mistake. From §8 on, 4–10 of 120 mistakes reached the memory per measurement, against

Table 2: All registered measurements, 40 fresh users each. L1: F−off in points on 160 fact probes (§12: the 116 that Sela-2 answered in all arms; off was 0 % in every run), with 95 % Wilson interval. L2, L2b: replies F / FK out of 280 per arm (§12: 360); off was 0 in every run. L3: % with 95 % cluster-bootstrap interval. L4: app prompts with a problem, F / FK (off). L5: writing speed FK/F. L6: % of Sela-2's probe replies (291 per arm), F / FK. †See § 6.1.

	Users	L1 F−off [95 % CI]	L2	L2b	L3 [95 % CI]	L4	L5	L6	Decision
§7	v1	52.5 [44.8, 60.1]	0 / 6	7 / 7	+0.55 [−1.10, 2.12]	0 / 0 (2)	0.790	–	A+B off, C off
§8	v2	35.0 [28.0, 42.7]	0 / 0	0 / 0	+1.45 [0.26, 2.81]	0 / 1 (2)	0.972	–	A+B on, C on
§9	v3 · labels	20.0 [14.5, 26.9]	0 / 1	0 / 0	+0.78 [−0.39, 2.32]	1 / 2 (1)	0.963	–	C off again
§9	v3 · sentences	44.4 [36.9, 52.1]	0 / 0	0 / 0	+0.78 [−0.39, 2.31]	0 / 0 (1)	0.968	–	not adopted†
§10	v4 · labels	23.8 [17.8, 30.9]	0 / 0	0 / 0	+0.09 [−0.59, 1.17]	3 / 1 (2)	0.969	–	–
§10	v4 · sentences	34.4 [27.5, 42.0]	0 / 2	0 / 0	+0.09 [−0.58, 1.16]	0 / 0 (2)	0.966	–	sentences adopted
§11	real · test	40.0 [32.7, 47.7]	0 / 1	0 / 0	+1.59 [0.50, 2.91]	0 / 1 (2)	0.967	–	C on
§12	real · test2, filter	30.2 [22.6, 39.1]	0 / 0	0 / 0	+3.07 [1.19, 5.35]	2 / 1 (2)	0.965	0.0 / 1.0	filter adopted
§12	real · test2, all	30.2 [22.6, 39.1]	0 / 0	0 / 0	+3.07 [1.18, 5.27]	1 / 1 (2)	0.975	24.7 / 29.9	comparison

Table 3: Fate of the 120 injected mistakes per registered measurement, read from the memory itself, and FK replies that contained an injected wrong form (280 per run; §12: 360). Both runs of §9, §10 and §12 teach the same memory; their copies are given as labels / sentences and filter on / off. Spell checking applies to English only.

Measurement	spell-checked	not learnable	filtered	stored	copied (FK)
§7 (v1)	37	0	60	23	6
§8 (v2)	34	0	76	10	0
§9 (v3)	33	0	81	6	1 / 0
§10 (v4)	37	4	74	5	0 / 2
§11 (real test)	33	11	70	6	1
§12 (real test2)	37	7	72	4	0 / 0

Table 4: Letter rules (excerpt; all 18 in Table 9): L3 on real development and consumed §7 users, mistakes stored over four sets, correct real-development sentences dropped (of 300). The earlier fix is the word-alone check of commit 63123a5.

Rule	L3 real dev (%)	L3 §7 (%)	stored	dropped
previous rule, no outside check	1.99	3.24	45	119
+ shape rule (chosen)	2.03	3.37	45	116
word alone > 2.08 nats (earlier fix)	1.97	3.00	81	99
fixed sentence > 1.5 nats + shape rule	2.18	3.14	66	95

23 with the first filter. On the §7 users used for development, the code measured in §8 had stored none (App. D): a gap between development and test for a filter whose thresholds were set on those users. The leaks are specific. A probe of the final code on the real development users, which unlike the app does not spell-check English, stored 14/120 mistakes: 9 English misspellings that the app's spell checker corrects first (*definately*, *freind*) and 5 of 60 German mistakes. For *Packet* and one *wiederrum* the correct forms made the message 2.01 and 2.07 nats likelier, just short of $\log 8 \approx 2.08$; the other *wiederrum* was never checked because its message held three more surprising words; *Rhytmus* lacks a letter, which is not tried; and for *garnicht* the split made that message less likely. F replies never contained an injected mistake. Of the 2,400 FK replies of §8–§12, four did: two reproduced a stored *garnicht*, while in the other two the memory had filtered the sentence and *ich habe gegangen* arose without a stored copy.

6.3 Why the memory's gain fell: lexical reuse

The memory gained +3.39 % on the §7 users during development but only +1.45, +0.78 and +0.09 % on the three fresh synthetic groups (Fig. 2a). The mixing setting does not explain this: on the consumed §9 and §10 users, the best of 60 settings of $(\lambda_{\max}, \theta, \tau)$ reached +0.83 and +0.25 %, against +0.76 and +0.05 % for the app's setting, and larger memory shares only hurt, down to −3.41 % (Table 6 and Fig. 4).

The evidence points to the test sentences: a memory of someone's wording can only help where they reuse it. For every held-out sentence we measured the share of its lower-cased word bigrams that already occur in the same user's taught messages. Pooled per group, this share was 10.5 % for the §7 users and 5.3, 4.6 and 2.7 % for the three test groups, the same order as their gains. Over all 320 synthetic sentence scores, the gain rises with reuse (Fig. 2b, Table 7) from +0.36 % for sentences sharing no word pair with the user's messages to +5.34 % for those sharing a fifth or more; the per-sentence rank correlation is $\rho = 0.42$ (95 % CI [0.24, 0.57]). The pattern holds within three of four groups: sentences with reuse gained +4.16, +2.30 and +1.71 %, those without +0.54, +0.78 and +0.23 % (§7, §8, §9); in §10 the 17 sentences with reuse gained nothing (−0.12 %). How much the held-out sentences reused the teaching messages was our choice when writing them, not a property of users, and with only 10 hobby scripts per group a few scripts decided it.

Real users reuse more of their wording: on the 40 real development users, 12.9 % of the held-out messages' word pairs occurred in their earlier messages (recorded in the preregistration). On the deciding test with 40 real users, the memory's gain was +1.59 % (95 % CI [0.50, 2.91]), and it met all criteria of layer C: one copied mistake (limit 2), no role confusion, writing speed 0.967 of F. It was switched on. On 40 further real users (§12, where it was reported but not decided on) it gained +3.07 % ([1.19, 5.35]). The intervals are wide: 40 users cannot establish with confidence that the deciding gain of 1.59 % lies above the 1 % threshold.

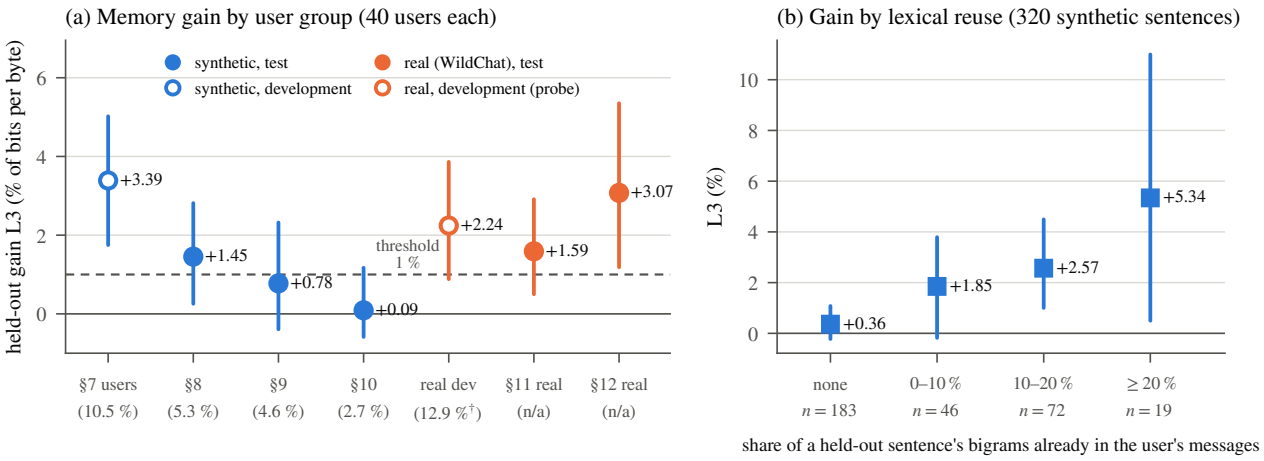


Figure 2: (a) Memory gain L3 on held-out messages by user group, with 95 % bootstrap intervals (over hobby scripts for synthetic groups, over users for real groups). In parentheses: share of the held-out sentences' word pairs (bigrams) already present in the same user's taught messages. †Recorded in the preregistration (§11); the texts are not in the repository, so it cannot be recomputed here. The real development value comes from a development probe, the others from app measurements. (b) Gain over the 320 synthetic held-out sentence scores, binned by bigram reuse; cluster-bootstrap intervals over hobby scripts.

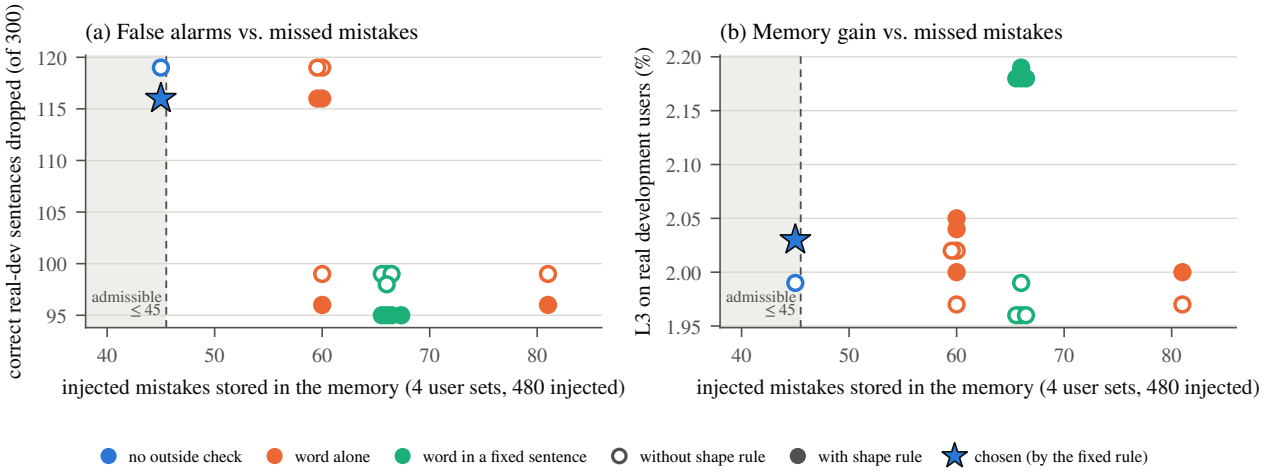


Figure 3: The 18 letter rules of development probe round 14, replayed from the same traces. Horizontal: injected mistakes stored, summed over four user sets (480 injected). (a) Correct real-development sentences dropped whole (lower is better). (b) Memory gain on the real development users (higher is better; note the narrow range). Rules left of the dashed line were admissible under the criterion fixed in advance; the star marks the selected rule. Rules with identical results are drawn slightly apart. The probe does not run the app's English spell checker, so stored counts include English misspellings that the app corrects first.

6.4 Choosing the letter rule

Figure 3 shows the trade-off in the letter check. The fixed-sentence checks at every threshold and the word-alone checks from 1.5 nats reduced false alarms, from 116–119 to 95–99 correct sentences dropped on the real development users, but every outside check let more injected mistakes through: 60–81 stored against 45 without one (Table 4). Only the two rules without an outside check were admissible, and the criterion selected the shape rule, which stores no more mistakes and drops fewer correct sentences (116 instead of 119). The earlier counter-measure (word alone, above 2.08 nats) stored 81. The gains of all 18 rules lie within 0.23 points, far less than the uncertainty of L3; the choice was made on mistakes, as intended.

The filter as a whole has a price. In a probe of the final code on the real development users (round 15, in which a mistake sentence no longer merges with a real message that lacks closing punctuation, so that 353 correct sentences are counted instead of 300), it dropped 141/353 correct sentences whole, against 80/353 with word-level filtering alone, and the memory gained 2.24 % instead of 3.21 %; in exchange, the error-carrying sentences that still contributed tokens fell from 106/120 to 52/120. Not learning a user's mistakes cost about 30 % of what the memory could gain from those users.

6.5 Unrequested facts and the relevance filter

L1 counts every mention of a fact, whether it fits or not, and many did not fit: asked what to do with the dog in a free afternoon, Sela-2 answered *Niklas und Bruno machen gerne*

Table 5: §12 on 40 further real test users with blind-written fact probes: relevance filter on (decides) and off (comparison on the same users, probes and seeds). Counts are replies; 95 % Wilson intervals in brackets.

Criterion	Filter on	Filter off	Threshold
L1 fact use, F (off: 0 in both runs)	35/116 (30.2) [22.6, 39.1]	35/116 (30.2) [22.6, 39.1]	F−off ≥ +15
with C (FK)	36/116 (31.0)	42/116 (36.2)	–
L6 unasked facts in probe replies, F	0/291 (0.0) [0.0, 1.3]	72/291 (24.7) [20.1, 30.0]	≤ 10 %
with C (FK)	3/291 (1.0) [0.4, 3.0]	87/291 (29.9) [24.9, 35.4]	–
L6 app replies naming the taught user's facts, F	1 of 96	12 of 96	≤ 3
with C (FK)	2 of 96	14 of 96	–
L2 copied mistakes, F / FK	0 / 0	0 / 0	≤ off +2
L2b user fact claimed as own, F / FK	0 / 0	0 / 0	≤ off +2
L4 app prompts with a problem, F / FK (off)	2 / 1 (2)	1 / 1 (2)	≤ off +2

gemeinsam Lasagne ("Niklas and Bruno like making lasagne together"). Counted afterwards on consumed measurements, 35.8–40.7 % of Sela-2's probe replies in arm F named a fact the probe had not asked for, and 10–15 of the 95–96 app replies named the taught user's facts, often as Sela-2's own ("Hi!" → "Hi! I like to play with my dog Rex."; Table 8). Without the note the rate was zero. Large assistants show the same failure [16, 22].

§12 tested a deliberately simple remedy: the note shows only the facts the message is about. A fact qualifies if the message names its value (*Rex*, *Leeds*), or its subject through short cue lists per category and language (*cook*, *dinner*, *Rezept* for the dish; *dog*, *pet*, *Hund* for the pet; *sign-off*, *e-mail*, *Gruß-formel* for the name; *near me*, *in der Nähe* for the city), or if it asks what Sela-2 knows about the user (then all are shown). The role-confusion guard still checks all facts. The cue words were frozen in a commit (96e9897) before the test probes existed. The probes were then written by a separate AI agent instance from a description of the four fact intents and of neutral requests, without access to the repository, the filter or its cue words: six formulations per fact and twelve neutral requests per language, committed with the registration. Each of 40 further real test users received one blind probe per fact and two neutral requests. We read the probes after they arrived and changed nothing. Measurement 1 (filter on) decided; measurement 2 (filter off) ran on the same users, probes and seeds.

With the filter, Sela-2 used the facts exactly as often where they were asked for (35/116 fact probes in both runs) but almost never volunteered them: 0/291 probe replies named an unrequested fact, against 72/291 without the filter, and 1 against 12 of 96 app replies (Table 5). The filter met every registered criterion and became the default. Two qualifications. First, the app count of L6 was recorded for every reply during the measurement, but the app evaluation's summary did not add it up; we counted it afterwards from the same recorded replies, with no new measurement, and only that entry changed (commit 2edd391). Second, cue words fail in both directions: the remaining app cases (one prompt in F, two in FK) were requests to summarise a text that was not about the user, in which a cue word showed a fact (*email* the name, German *Stadt* the city), and blind English probes without a cue word received no note, so English fact use was one probe lower with the filter (8 against 9 of 36; German 27 against 26 of 80). On the blind probes, ENGRAMM answered 27.5 % of the fact probes itself, so L1 rests on the 116 that Sela-2 answered in all arms.

Cost. The memory slowed writing by 2–4 %: L5 lay between 0.963 and 0.975 in all registered measurements from §8 on (0.79 for the first version, under an older definition that included reading the prompt). The analysis runs after the reply and blocks a new reply for at most one forward pass. Eight teaching messages produced on average 54–59 memory entries per user from §8 on (78 with the first filter), each a 128-dimensional key (cached on disk in half precision) and a token id.

7 Discussion

What we think generalises. A personal memory needs a policy for what it may learn, not only for what it retrieves. A model's own likelihood, compared on whole texts, is a cheap, language-independent basis for such a policy and catches what token-level checks cannot (merged and split words). Its price is real: on our real development users, the filter discarded two fifths of the correct sentences and about 30 % of the achievable gain. Where to draw the line is a product decision that a criterion fixed in advance can make explicit, as for our letter rule. Second, showing a small model every fact it knows makes it talk about the user; choosing facts by relevance before generation solved this here with simple, frozen cue words.

Lessons about synthetic test users. Synthetic users are convenient and safe, and we still recommend them for early development, but they misled us in three ways. Properties that decide the outcome can be set unknowingly by whoever writes the data: our choice of how much held-out sentences reused the users' wording, not the method, most likely drove the decline from +3.39 % to +0.09 %; such properties should be measured and reported with the results. "40 users" may hide far fewer independent units: our synthetic groups had 10 hobby scripts each, and resampling users instead of scripts would have shown intervals only about half as wide (the script intervals are 1.9–2.0 times wider). And the development set shapes the method: zero stored mistakes on the §7 development users became 4–10 on each fresh group. A modest sample of real users changed our conclusion about layer C.

Lessons about preregistration. The registrations prevented us from rescuing the sentence note in §9 after the fact, although its advantage was large, and they made the repetition in §10 necessary and convincing; they also exposed our own errors. The §9 rule shows the design lesson: a decision should depend only on the criteria of the component it decides. Preregistration does not make small samples large: several decisions passed or failed a point threshold by less than the uncertainty of 40 users, and a registered rule makes such decisions consistent, not certain.

8 Limitations

- **One model, one application, two languages.** All results concern Sela-2 (mix30) inside ENGRAMM in English and German; a larger model might use facts better and copy fewer mistakes.
- **Small samples.** 40 users per measurement; the synthetic groups contain only 10 independent hobby scripts. Several decisions rested on point estimates close to their thresholds.
- **Synthetic facts and mistakes.** Even for real users, facts and mistakes were synthetic: four fact statements and three appended sentences from one fixed list of 24 mistakes, the same list in every group. The filter's thresholds were developed on the mistake types later tested, and we did not measure how many of the real users' own, natural mistakes entered the memory.
- **What L1 and L3 do not measure.** L1 counts mentions of facts, not whether they help; L6 covers one way of being unhelpful. L3 measures how well the model predicts a user's own held-out messages, which need not mean better replies [56]. We did not measure answer quality or helpfulness, and no human rated any reply.
- **The filter's errors.** It discards many correct sentences and still stores some mistakes; grammar mistakes of correct words, missing letters and mistakes that are valid words in the other language largely escape it.
- **Cue words** were frozen before the blind probes existed but miss phrasings and misfire on requests that are not about the user, as both happened in §12.
- **Our own mistakes.** The §9 rule coupled unrelated criteria; a development counter-measure (the word-alone check) let real mistakes through before the letter-rule comparison replaced it; the §12 app intrusion count was summed only after the measurement; and some counts in the project's prose documents differ from the result files (App. G). We report the result files throughout.
- **Speed of the process.** The whole sequence took one day, with most implementation done by an AI coding agent (§ 9). Registrations precede their measurements in the version history, but no external party reviewed them before the measurements.

9 Ethics, privacy and AI assistance

Real user data. WildChat-1M consists of conversations with a chatbot whose users consented to their collection for research [64]; it is released under the ODC-By licence and identifies users only by hashed IP addresses. We used English and German conversations not flagged as toxic or redacted. The repository stores only the selection (conversation hashes, turn identifiers, SHA-256 and length of each message); the texts are reassembled from the fixed public revision inside the CI job of a measurement and checked against their hashes. Result files keep real messages only as hash and length, and every run of four or more words written by a real user is removed from stored replies and facts. This paper quotes no text written by a real user: quoted user messages and facts are synthetic or blind-written probes, and quoted replies are Sela-2's.

The system. What layers A, B and C learn stays on the user's computer and can be deleted exactly. The relevance filter and the guards reduce, but do not eliminate, the risk that personal facts surface in unexpected contexts. Sela-2's weights are not distributed; parts of its training data were generated by models whose licences impose conditions on redistribution, which would first have to be met.

AI assistance. The code, the measurement workflows and most of the documentation in the repository were written by an AI coding agent (Claude, by Anthropic) under the author's direction: up to the commit that applied the last decision (68a2782), 806 of 859 commits are attributed to the agent and 49 to automated CI workflows. The code reviews of § 5 and the blind probes of §12 (written without access to the repository) were also made by AI agents: separate instances of the same model, which had not written the code. The author set the goals, the constraints and the budget and is responsible for the claims. This paper was drafted with AI assistance from the repository's records; every number in it is computed from, or quoted from, repository files (App. A).

10 Conclusion

A small language model trained from scratch for about USD 23 can learn from its user on the user's own computer: a note of the user's facts, shown only when the message is about them, and a memory of the user's wording guarded by the model's own judgement of what looks like a mistake. Measured under preregistration on fresh users, the facts were used, the user's mistakes were rarely stored and almost never repeated, and the memory helped real users modestly. The most transferable result concerns evaluation: a memory-based personalisation can appear to stop working simply because the test sentences written for it stopped reusing the users' words, and only real users revealed this. Preregistration did not make 40 users enough to settle every question, but it made each answer, and each of our mistakes, visible.

Reproducibility statement. Every number is computed by `paper/make_figures.py` from result files in the project repository or quoted from them (App. A). Rerunning the measurements requires the Sela-2 weights (`sela2-mix30.npz`, not public), ENGRAMM's lite knowledge pack, the GitHub Actions workflows `sela2_learn_eval.yml` and `sela2_learn_probe.yml`, and for real users the public WildChat-1M revision 7d6490e. Synthetic users are generated deterministically by `experiments/sela2_learn_data.py`; the real-user selection is in `data/sela2/learn_real_ids.json`. Code and weights are available from the author on request.

References

[1] N. Amsel, D. Persson, C. Musco, and R. M. Gower. The Polar Express: Optimal matrix sign methods and their application to the Muon algorithm. In *International Conference on Learning Representations (ICLR)*, 2026. arXiv:2505.16932.

[2] Y. Belinkov and Y. Bisk. Synthetic and natural noise both break neural machine translation. In *International Conference on Learning Representations (ICLR)*, 2018.

[3] L. Ben Allal et al. SmolLM2: When smol goes big – data-centric training of a small language model, 2025.

[4] L. Bertinetto, J. F. Henriques, S. Albanie, M. Paganini, and G. Varol, editors. *NeurIPS 2020 Workshop on Pre-registration in Machine Learning*, volume 148 of *Proceedings of Machine Learning Research*, 2021. PMLR.

[5] S. Biderman, H. Schoelkopf, Q. Anthony, H. Bradley, K. O'Brien, E. Hallahan, M. A. Khan, S. Purohit, U. S. Prashanth, E. Raff, A. Skowron, L. Sutawika, and O. van der Wal. Pythia: A suite for analyzing large language models across training and scaling. In *Proceedings of the 40th International Conference on Machine Learning (ICML)*, volume 202 of *Proceedings of Machine Learning Research*, pages 2397–2430, 2023.

[6] Y. Bisk, R. Zellers, R. Le Bras, J. Gao, and Y. Choi. PIQA: Reasoning about physical commonsense in natural language. In *Proceedings of the AAAI Conference on Artificial Intelligence*, volume 34, pages 7432–7439, 2020. doi: 10.1609/aaai.v34i05.6239.

[7] S. Borgeaud, A. Mensch, J. Hoffmann, et al. Improving language models by retrieving from trillions of tokens. In *Proceedings of the 39th International Conference on Machine Learning (ICML)*, volume 162 of *Proceedings of Machine Learning Research*, pages 2206–2240, 2022.

[8] L. Bourtoule, V. Chandrasekaran, C. A. Choquette-Choo, H. Jia, A. Travers, B. Zhang, D. Lie, and N. Papernot. Machine unlearning. In *2021 IEEE Symposium on Security and Privacy (SP)*, pages 141–159, 2021. doi: 10.1109/SP40001.2021.00019.

[9] H. Brown, K. Lee, F. Mireshghallah, R. Shokri, and F. Tramèr. What does it mean for a language model to preserve privacy? In *Proceedings of the 2022 ACM Conference on Fairness, Accountability, and Transparency (FAccT)*, 2022.

[10] C. Bryant and T. Briscoe. Language model based grammatical error correction without annotated training data. In *Proceedings of the Thirteenth Workshop on Innovative Use of NLP for Building Educational Applications (BEA)*, pages 247–253, 2018. doi: 10.18653/v1/W18-0529.

[11] C. Bryant, Z. Yuan, M. R. Qorib, H. Cao, H. T. Ng, and T. Briscoe. Grammatical error correction: A survey of the state of the art. *Computational Linguistics*, 49(3):643–701, 2023. doi: 10.1162/coli_a_00478.

[12] N. Carlini, F. Tramèr, E. Wallace, M. Jagielski, A. Herbert-Voss, K. Lee, A. Roberts, T. Brown, D. Song, Ú. Erlingsson, A. Oprea, and C. Raffel. Extracting training data from large language models. In *30th USENIX Security Symposium*, 2021.

[13] P. Clark, I. Cowhey, O. Etzioni, T. Khot, A. Sabharwal, C. Schoenick, and O. Tafjord. Think you have solved question answering? Try ARC, the AI2 reasoning challenge, 2018.

[14] A. Drozdov, S. Wang, R. Rahimi, A. McCallum, H. Zamani, and M. Iyyer. You can't pick your neighbors, or can you? When and how to rely on retrieval in the kNN-LM. In *Findings of the Association for Computational Linguistics: EMNLP 2022*, pages 2997–3007, 2022. doi: 10.18653/v1/2022.findings-emnlp.218.

[15] B. Efron and R. J. Tibshirani. *An Introduction to the Bootstrap*, volume 57 of *Monographs on Statistics and Applied Probability*. Chapman & Hall, New York, 1993.

[16] X. Feng, W. Gan, X. Chen, Q. Dai, and Y. Liu. How does personalized memory shape LLM behavior? Benchmarking rational preference utilization in personalized assistants, 2026.

[17] A. Gelman and E. Loken. The statistical crisis in science. *American Scientist*, 102(6):460, 2014. doi: 10.1511/2014.111.460.

[18] E. Grave, A. Joulin, and N. Usunier. Improving neural language models with a continuous cache. In *International Conference on Learning Representations (ICLR)*, 2017.

[19] S. Gururangan, S. Swayamdipta, O. Levy, R. Schwartz, S. Bowman, and N. A. Smith. Annotation artifacts in natural language inference data. In *Proceedings of the 2018 Conference of the North American Chapter of the Association for Computational Linguistics (NAACL)*, pages 107–112, 2018. doi: 10.18653/v1/N18-2017.

[20] A. Hard, K. Rao, R. Mathews, S. Ramaswamy, F. Beaufays, S. Augenstein, H. Eichner, C. Kiddon, and D. Ramage. Federated learning for mobile keyboard prediction, 2018.

[21] A. Henry, P. R. Dachapally, S. S. Pawar, and Y. Chen. Query-key normalization for transformers. In *Findings of the Association for Computational Linguistics: EMNLP 2020*, pages 4246–4253, 2020. doi: 10.18653/v1/2020.findings-emnlp.379.

[22] Y. Hu, Z. Long, J. Guo, X. Sui, X. Fu, W. Zhao, Y. Zhao, and B. Qin. OP-Bench: Benchmarking over-personalization for memory-augmented personalized conversational agents, 2026.

[23] W. B. Johnson and J. Lindenstrauss. Extensions of Lipschitz mappings into a Hilbert space. In *Conference in Modern Analysis and Probability*, volume 26 of *Contemporary Mathematics*,

pages 189–206. American Mathematical Society, 1984. doi: 10.1090/conm/026/737400.

[24] K. Jordan, Y. Jin, V. Boza, J. You, F. Cesista, L. Newhouse, and J. Bernstein. Muon: An optimizer for hidden layers in neural networks. https://kellerjordan.github.io/posts/muon/, 2024.

[25] K. Jordan et al. modded-nanogpt: Speedrunning the NanoGPT baseline. https://github.com/KellerJordan/modded-nanogpt, 2024.

[26] A. Karpathy. nanoGPT. https://github.com/karpathy/nanoGPT, 2022.

[27] A. Karpathy. nanochat: The best ChatGPT that $100 can buy. https://github.com/karpathy/nanochat, 2025.

[28] M. D. Kernighan, K. W. Church, and W. A. Gale. A spelling correction program based on a noisy channel model. In *Proceedings of the 13th International Conference on Computational Linguistics (COLING), Volume 2*, pages 205–210, 1990.

[29] U. Khandelwal, O. Levy, D. Jurafsky, L. Zettlemoyer, and M. Lewis. Generalization through memorization: Nearest neighbor language models. In *International Conference on Learning Representations (ICLR)*, 2020.

[30] U. Khandelwal, A. Fan, D. Jurafsky, L. Zettlemoyer, and M. Lewis. Nearest neighbor machine translation. In *International Conference on Learning Representations (ICLR)*, 2021.

[31] M. King and P. Cook. Evaluating approaches to personalizing language models. In *Proceedings of the Twelfth Language Resources and Evaluation Conference (LREC)*, pages 2461–2469, 2020.

[32] A. Köpf, Y. Kilcher, D. von Rütte, S. Anagnostidis, et al. OpenAssistant conversations – democratizing large language model alignment. In *Advances in Neural Information Processing Systems 36 (NeurIPS 2023), Datasets and Benchmarks Track*, 2023.

[33] B. Krause, E. Kahembwe, I. Murray, and S. Renals. Dynamic evaluation of neural sequence models. In *Proceedings of the 35th International Conference on Machine Learning (ICML)*, volume 80 of *Proceedings of Machine Learning Research*, pages 2766–2775, 2018.

[34] R. Kuhn and R. De Mori. A cache-based natural language model for speech recognition. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 12(6):570–583, 1990. doi: 10.1109/34.56193.

[35] P. Lewis, E. Perez, A. Piktus, F. Petroni, V. Karpukhin, N. Goyal, H. Küttler, M. Lewis, W.-t. Yih, T. Rocktäschel, S. Riedel, and D. Kiela. Retrieval-augmented generation for knowledge-intensive NLP tasks. In *Advances in Neural Information Processing Systems 33 (NeurIPS 2020)*, 2020.

[36] J. Li, A. Fang, G. Smyrnis, M. Ivgi, M. Jordan, et al. DataComp-LM: In search of the next generation of training sets for language models. In *Advances in Neural Information Processing Systems 37 (NeurIPS 2024), Datasets and Benchmarks Track*, 2024.

[37] Z. Li, L. Liu, C. Liang, W. Chen, and T. Zhao. NorMuon: Making Muon more efficient and scalable, 2025.

[38] B. Messmer, V. Sabolčec, and M. Jaggi. Enhancing multilingual LLM pretraining with model-based data selection, 2025.

[39] T. Mihaylov, P. Clark, T. Khot, and A. Sabharwal. Can a suit of armor conduct electricity? A new dataset for open book question answering. In *Proceedings of the 2018 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, pages 2381–2391, 2018. doi: 10.18653/v1/D18-1260.

[40] B. A. Nosek, C. R. Ebersole, A. C. DeHaven, and D. T. Mellor. The preregistration revolution. *Proceedings of the National Academy of Sciences*, 115(11):2600–2606, 2018. doi: 10.1073/pnas.1708274114.

[41] B. A. Olken. Promises and perils of pre-analysis plans. *Journal of Economic Perspectives*, 29(3):61–80, 2015. doi: 10.1257/jep.29.3.61.

[42] C. Packer, S. Wooders, K. Lin, V. Fang, S. G. Patil, I. Stoica, and J. E. Gonzalez. MemGPT: Towards LLMs as operating systems, 2023.

[43] J. S. Park, J. C. O'Brien, C. J. Cai, M. R. Morris, P. Liang, and M. S. Bernstein. Generative agents: Interactive simulacra of human behavior. In *Proceedings of the 36th Annual ACM Symposium on User Interface Software and Technology (UIST)*, 2023. doi: 10.1145/3586183.3606763.

[44] G. Penedo, H. Kydlíček, L. Ben Allal, A. Lozhkov, M. Mitchell, C. Raffel, L. Von Werra, and T. Wolf. The FineWeb datasets: Decanting the web for the finest text data at scale. In *Advances in Neural Information Processing Systems 37 (NeurIPS 2024), Datasets and Benchmarks Track*, 2024.

[45] J. Pineau, P. Vincent-Lamarre, K. Sinha, V. Larivière, A. Beygelzimer, F. d'Alché Buc, E. Fox, and H. Larochelle. Improving reproducibility in machine learning research (a report from the NeurIPS 2019 reproducibility program). *Journal of Machine Learning Research*, 22(164):1–20, 2021.

[46] A. Radford, J. Wu, R. Child, D. Luan, D. Amodei, and I. Sutskever. Language models are unsupervised multitask learners. Technical report, OpenAI, 2019.

[47] J. W. Rae, A. Potapenko, S. M. Jayakumar, C. Hillier, and T. P. Lillicrap. Compressive transformers for long-range sequence modelling. In *International Conference on Learning Representations (ICLR)*, 2020.

[48] C. Raffel, N. Shazeer, A. Roberts, K. Lee, S. Narang, M. Matena, Y. Zhou, W. Li, and P. J. Liu. Exploring the limits of transfer learning with a unified text-to-text transformer. *Journal of Machine Learning Research*, 21(140):1–67, 2020.

[49] P. Rajpurkar, R. Jia, and P. Liang. Know what you don't know: Unanswerable questions for SQuAD. In *Proceedings of the 56th Annual Meeting of the Association for Computational Linguistics (ACL), Volume 2: Short Papers*, pages 784–789, 2018. doi: 10.18653/v1/P18-2124.

[50] K. Sakaguchi, R. Le Bras, C. Bhagavatula, and Y. Choi. WinoGrande: An adversarial Winograd schema challenge at scale. In *Proceedings of the AAAI Conference on Artificial Intelligence*,

volume 34, pages 8732–8740, 2020. doi: 10.1609/aaai.v34i05.6399.

[51] A. Salemi, S. Mysore, M. Bendersky, and H. Zamani. LaMP: When large language models meet personalization. In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (ACL)*, pages 7370–7392, 2024. doi: 10.18653/v1/2024.acl-long.399.

[52] R. Sennrich, B. Haddow, and A. Birch. Neural machine translation of rare words with subword units. In *Proceedings of the 54th Annual Meeting of the Association for Computational Linguistics (ACL)*, pages 1715–1725, 2016. doi: 10.18653/v1/P16-1162.

[53] J. P. Simmons, L. D. Nelson, and U. Simonsohn. False-positive psychology: Undisclosed flexibility in data collection and analysis allows presenting anything as significant. *Psychological Science*, 22(11):1359–1366, 2011.

[54] D. R. So, W. Mańke, H. Liu, Z. Dai, N. Shazeer, and Q. V. Le. Primer: Searching for efficient transformers for language modeling. In *Advances in Neural Information Processing Systems 34 (NeurIPS 2021)*, 2021.

[55] J. Su, M. Ahmed, Y. Lu, S. Pan, W. Bo, and Y. Liu. RoFormer: Enhanced transformer with rotary position embedding. *Neurocomputing*, 568:127063, 2024. doi: 10.1016/j.neucom.2023.127063.

[56] S. Wang, Y. Song, A. Drozdov, A. Garimella, V. Manjunatha, and M. Iyyer. kNN-LM does not improve open-ended text generation. In *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, pages 15023–15037, 2023. doi: 10.18653/v1/2023.emnlp-main.929.

[57] E. B. Wilson. Probable inference, the law of succession, and statistical inference. *Journal of the American Statistical Association*, 22(158):209–212, 1927. doi: 10.1080/01621459.1927.10502953.

[58] M. Wortsman, G. Ilharco, J. W. Kim, M. Li, S. Kornblith, R. Roelofs, R. Gontijo Lopes, H. Hajishirzi, A. Farhadi, H. Namkoong, and L. Schmidt. Robust fine-tuning of zero-shot models. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 7959–7971, 2022.

[59] F. F. Xu, U. Alon, and G. Neubig. Why do nearest neighbor language models work? In *Proceedings of the 40th International Conference on Machine Learning (ICML)*, volume 202 of *Proceedings of Machine Learning Research*, pages 38325–38341, 2023.

[60] Z. Yang, P. Qi, S. Zhang, Y. Bengio, W. Cohen, R. Salakhutdinov, and C. D. Manning. HotpotQA: A dataset for diverse, explainable multi-hop question answering. In *Proceedings of the 2018 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, pages 2369–2380, 2018. doi: 10.18653/v1/D18-1259.

[61] R. Zellers, A. Holtzman, Y. Bisk, A. Farhadi, and Y. Choi. HellaSwag: Can a machine really finish your sentence? In *Proceedings of the 57th Annual Meeting of the Association for Computational Linguistics (ACL)*, pages 4791–4800, 2019. doi: 10.18653/v1/P19-1472.

[62] B. Zhang and R. Sennrich. Root mean square layer normalization. In *Advances in Neural Information Processing Systems 32 (NeurIPS 2019)*, 2019.

[63] S. Zhang, E. Dinan, J. Urbanek, A. Szlam, D. Kiela, and J. Weston. Personalizing dialogue agents: I have a dog, do you have pets too? In *Proceedings of the 56th Annual Meeting of the Association for Computational Linguistics (ACL)*, pages 2204–2213, 2018. doi: 10.18653/v1/P18-1205.

[64] W. Zhao, X. Ren, J. Hessel, C. Cardie, Y. Choi, and Y. Deng. WildChat: 1m ChatGPT interaction logs in the wild. In *International Conference on Learning Representations (ICLR)*, 2024.

[65] W. Zhong, L. Guo, Q. Gao, H. Ye, and Y. Wang. MemoryBank: Enhancing large language models with long-term memory. In *Proceedings of the AAAI Conference on Artificial Intelligence*, volume 38, pages 19724–19731, 2024. doi: 10.1609/aaai.v38i17.29946.

[66] J. Zhou, T. Lu, S. Mishra, S. Brahma, S. Basu, Y. Luan, D. Zhou, and L. Hou. Instruction-following evaluation for large language models, 2023.

A Provenance of tables and figures

Paths are relative to the repository root. "Script" is `paper/make_figures.py`; it reads only the files named, writes `paper/generated/` and `paper/figures/`, and uses a fixed bootstrap seed per quantity.

Item	Source files
Figure 1, Alg. 1, Eq. (1)	`engramm/lm/sela_learn.py`, `engramm/lm/sela_chat.py`, `engramm/app/server.py` (constants as at commit 68a2782)
§ 3.1	`docs/MODEL_CARD_SELA2.md`; `results/sela2/pretrain_meta.json`, `sft_meta.json`, `bench.json`, `contamination.json`, `int8.json`; `results/sela2/variants/selection.md`; `docs/PLAN_V5_FROM_SCRATCH.md` §12
§ 3.2 (speed)	`results/sela2/<run>/summary.json` (`decode_tokens_per_s`, arm F) of the runs in Table 2
Table 1, App. B	`docs/PREREG_SELA2.md` §7–§12; `experiments/sela2_learn_eval.py` (`summarize`)
Table 2	`results/sela2/<run>/summary.json`, `part_*.json`, `app_learn_off.json` for `learn`, `learn_v2`, `learn_v3_labels`, `learn_v3_sentences`, `learn_v4_labels`, `learn_v4_sentences`, `learn_real`, `learn_real2_relevant`, `learn_real2_all`; hobby scripts from `data/sela2/learn_sessions{,_v2,_v3,_v4}.json`; intervals: script
Table 3	the same `summary.json` (`errors_caught`) and `part_*.json` (FK replies; fate of each mistake per user); script
§ 6.2 (leaks)	`results/sela2/learn_probe/probe_r15_real_dev.json` (`summary.by_wrong`; per-mistake traces); language of each wrong form from `data/sela2/learn_sessions{,_v2,_v3,_v4}.json`; script
Figure 2a	`part_*.json` of `learn_dev_v2f`, `learn_v2`, `learn_v3_labels`, `learn_v4_labels`, `learn_real`, `learn_real2_relevant`; `results/sela2/learn_probe/probe_r15_real_dev.json`; reuse from `data/sela2/learn_sessions*.json`; real development reuse (12.9 %) quoted from `docs/PREREG_SELA2.md` §11
Figure 2b, Table 7	`part_*.json` of `learn_dev_v2f`, `learn_v2`, `learn_v3_labels`, `learn_v4_labels`; `data/sela2/learn_sessions{,_v2,_v3,_v4}.json`; script (reproduces the table of §11 of the preregistration)
Table 6, Fig. 4	`results/sela2/learn_probe/probe_v3.json`, `probe_v4.json`, `probe_r15_real_dev.json`; `probe_r14_{v1,v3,v4,real_dev}.json`
Figure 3, Tables 4 and 9	`results/sela2/learn_probe/probe_r14_{real_dev,v1,v3,v4}.json` (`summary.letter_rules`); filter price: `probe_r15_real_dev.json` (`summary.rules`)
Table 8	`part_*.json`, `app_learn_{facts,knn}.json` of `learn_v3_sentences`, `learn_v4_sentences`, `learn_real`; `data/sela2/learn_sessions_v{3,4}.json`; definition `intrusion` in `experiments/sela2_learn_eval.py`
Table 5	`results/sela2/learn_real2_relevant/`, `learn_real2_all/` (`summary.json`, `part_*.json`, `app_learn_*.json`); `data/sela2/probes_v12.json`; `docs/PREREG_SELA2.md` §12.1
Table 10	`results/sela2/learn_dev_v2{a,...,h}/summary.json`, `app_learn_off.json`
§ 5 (uncertainty), § 7 (interval widths)	`part_*.json` of the runs in Table 2 (L1 and L3 per user; hobby scripts from `data/sela2/learn_sessions*.json`); script
§ 6.5 (cost)	`summary.json` (L5) and `part_*.json` (`knn_entries`) of the runs in Table 2; key format: `engramm/lm/sela_learn.py`
Quoted examples	`results/sela2/learn/part_de00-de04.json` (§ 6.1); `results/sela2/learn_real/part_de00-de04.json` and `app_learn_facts.json` (§ 6.5); `results/sela2/learn_real2_relevant/app_learn_{facts,knn}.json` and `data/sela2/chat_eval_prompts.json` (remaining §12 cases); `docs/PREREG_SELA2.md` §7 (trip to Italy) and §9 (labels)
Table 11, AI assistance	`git log` of the repository

B Decision rules by section

Section	Registered rule (summary; full text in `docs/PREREG_SELA2.md`)
§7, §8	A+B on if L1 F−off ≥ +15 (valid coverage) and L2, L2b, L4 for F ≤ off +2. C on if A+B on, L3 ≥ 1 %, L2, L2b, L4 for FK ≤ off +2 and L5 ≥ 0.85 (§7: tokens per second including the prompt, FK/off; from §8: writing speed, FK/F).
§9	Sentence note becomes default if both runs are valid, the sentence run meets the §8 rule for A+B *and for C*, and its L1 exceeds the label run's by ≥ 5 points. The §8 decisions stand unless the label run contradicts them.
§10	Sentence note becomes default if both runs are valid, its L1 exceeds the label run's by ≥ 5 points, and it meets the A+B criteria of §8. Criteria of C reported only.
§11	C on if L3 ≥ 1 %, L2, L2b and L4 for FK ≤ off +2, and L5 ≥ 0.85; A+B criteria reported only.
§12	Relevance filter becomes default if measurement 1 is valid and meets L1 F−off ≥ +15, L2, L2b, L4 for F ≤ off +2, L6 F ≤ 10 % and L6 app F ≤ 3. C stays on; its criteria are reported.

C Additional tables

Table 6: L3 (%) on held-out messages under the app's mixing setting $(\lambda_{\max}, \theta, \tau) = (0.1, 0.9, 0.01)$, and the best and worst of 60 settings (development probes on consumed and development users).

Users	Probe file	app setting	best of 60 (setting)	worst of 60 (setting)
§9 users	`probe_v3.json`	+0.76	+0.83 (0.05, 0.9, 0.01)	−2.55 (0.3, 0.7, 0.05)
§10 users	`probe_v4.json`	+0.05	+0.25 (0.05, 0.9, 0.01)	−3.41 (0.3, 0.7, 0.05)
real dev users (round 15)	`probe_r15_real_dev.json`	+2.24	+2.24 (0.1, 0.9, 0.01)	−1.58 (0.3, 0.7, 0.05)

Table 7: Memory gain by lexical reuse over the 320 synthetic held-out sentence scores of §7 (development), §8, §9 and §10, with 95 % intervals from a cluster bootstrap over the 40 hobby scripts (80 distinct sentences).

Bigrams already in the user's messages	Scores	L3 (%)	95 % CI
no shared bigram	183	+0.36	[−0.22, 1.08]
0–10 %	46	+1.85	[−0.18, 3.79]
10–20 %	72	+2.57	[1.00, 4.49]
≥ 20 %	19	+5.34	[0.50, 10.99]

Table 8: Unrequested facts before §12, counted after §11 on consumed measurements (note listing every fact): Sela-2 probe replies naming a user fact the probe did not ask for, and app replies (after teaching the first test user) naming that user's facts.

Measurement	off	F (%)	FK (%)	app F / FK
§9 sentences (`learn_v3_sentences`)	0/267	97/267 (36.3)	91/267 (34.1)	10 / 12 of 95
§10 sentences (`learn_v4_sentences`)	0/268	96/268 (35.8)	89/268 (33.2)	15 / 12 of 96
§11 real test (`learn_real`)	0/273	111/273 (40.7)	115/273 (42.1)	10 / 13 of 96

Table 9: All 18 letter rules of development probe round 14 (L3 in %; stored: injected mistakes over the four sets, 480 injected; dropped: correct real-development sentences dropped whole, of 300). ⋆Selected.

Rule	L3 real dev	L3 §7	L3 §9	L3 §10	stored	dropped
no outside check	1.99	3.24	0.74	0.06	45	119
no outside check + shape rule⋆	2.03	3.37	0.74	0.06	45	116
word alone > 0.00 nats	2.02	3.23	0.74	0.07	60	119
word alone > 0.00 nats + shape rule	2.05	3.37	0.74	0.07	60	116
word alone > 1.00 nats	2.02	3.23	0.74	0.08	60	119
word alone > 1.00 nats + shape rule	2.04	3.37	0.74	0.08	60	116
word alone > 1.50 nats	1.97	3.01	0.73	0.03	60	99
word alone > 1.50 nats + shape rule	2.00	3.14	0.73	0.03	60	96
word alone > 2.08 nats	1.97	3.00	0.72	0.03	81	99
word alone > 2.08 nats + shape rule	2.00	3.13	0.72	0.03	81	96
fixed sentence > 0.00 nats	1.96	3.14	0.74	0.03	66	99
fixed sentence > 0.00 nats + shape rule	2.19	3.14	0.74	0.03	66	95
fixed sentence > 1.00 nats	1.96	3.14	0.74	0.03	66	99
fixed sentence > 1.00 nats + shape rule	2.18	3.14	0.74	0.03	66	95
fixed sentence > 1.50 nats	1.96	3.14	0.74	0.03	66	99
fixed sentence > 1.50 nats + shape rule	2.18	3.14	0.74	0.03	66	95
fixed sentence > 2.08 nats	1.99	3.14	0.74	0.03	66	98
fixed sentence > 2.08 nats + shape rule	2.18	3.14	0.74	0.03	66	95

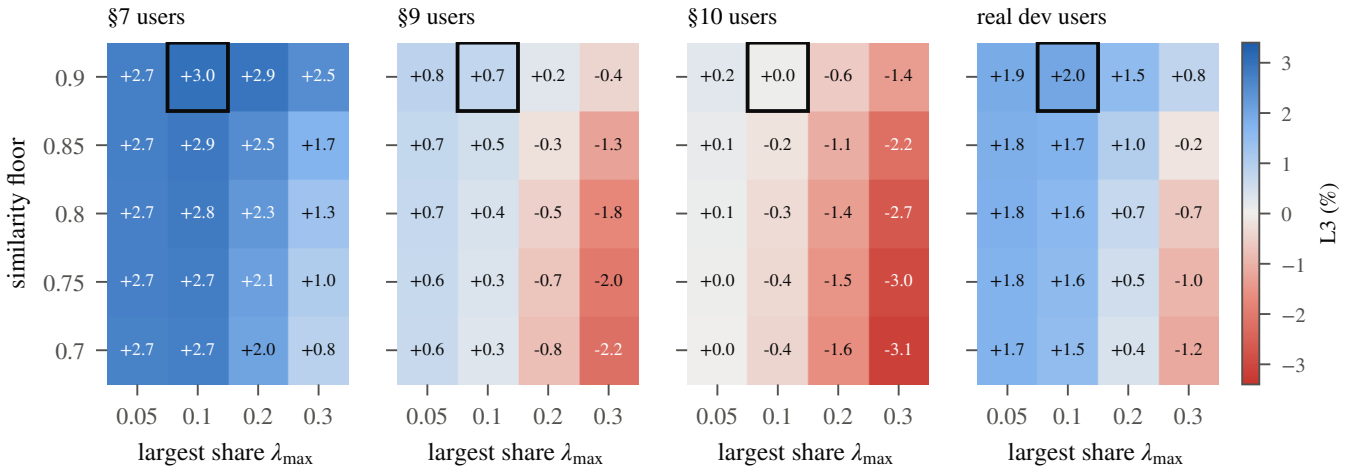


Figure 4: L3 (%) over the mixing grid at $\tau = 0.01$ (development probe round 14, one code state for all four user sets). The framed cell is the app's setting.

D Development runs on the §7 users

Table 10: App runs with all three arms on the consumed §7 users while developing the second version and the note forms (not used for any decision). L1 in points; L2, L2b and L4 as in Table 2; "stored": injected mistakes that reached the memory. [a]Older definition of L5 (tokens per second including the prompt, FK/off).

Run	Results commit	L1	L2 F/FK	L2b F/FK	L3 (%)	L4 F/FK (off)	L5	stored
v2a	667b7e7	30.6	0 / 6	0 / 0	+0.55	2 / 3 (2)	0.904[a]	23
v2b	bff5750	30.6	0 / 1	0 / 0	+2.91	2 / 1 (2)	0.962	5
v2c	0b6afaf	30.6	0 / 0	0 / 0	+0.53	2 / 1 (2)	0.976	0
v2d	23a77e6	30.6	0 / 1	0 / 0	+3.27	2 / 1 (2)	0.963	0
v2e	6b78c5a	30.6	0 / 0	0 / 0	+3.39	2 / 0 (2)	0.975	0
v2f	f8300cb	30.6	0 / 0	0 / 0	+3.39	2 / 0 (2)	0.979	0
v2g	3f69535	28.8	0 / 0	0 / 0	+3.39	2 / 0 (2)	0.960	0
v2h	0bc896b	40.0	0 / 0	0 / 0	+3.39	1 / 0 (2)	0.968	0

v2a: note and guard with the old memory filter; v2b: whole-text word check and the grid's mixing setting; v2c: sentence exclusion on any near-form flag (too strict: L3 fell to 0.53 %); v2d: sentence exclusion only for confirmed mistakes; v2e: similarity floor 0.90; v2f: fixes from the third review (the code measured in §8); v2g: guard against printing the note; v2h: note as sentences.

E Selection of real users

Source: `allenai/WildChat-1M`, revision `7d6490e`, the first three of 14 parquet shards. Conversations labelled English or German and not marked toxic or redacted; user turns in the conversation's language. A user is a hashed IP address within one language. Messages are kept if they are plain prose (20–300 characters; no code, links or markup; at most two line breaks; at least 85 % letters and spaces), ordered by conversation time and then turn identifier, and taken only if not a near copy of an earlier taken message (`difflib` similarity < 0.8). Users with six such messages qualify: four taught, two held out. Split: development if the first byte of SHA-256(hashed IP) is even, otherwise test. Qualified pools: development 1,504 English and 58 German users; test 1,500 and 54. With seed 20261023, 20 users per language were drawn for development and for test (§11); for §12, 20 per language from the 1,478 English and 34 German test-side users whose hashed IP was in no earlier set (seed 20261025). No user appears in two sets. The selection file holds, per message, the shard, conversation hash, turn identifier, SHA-256 and length, and was committed (41431d1; §12: 789abea) before any measurement on these users. Facts were drawn from the pools of the synthetic generator; mistakes and probes follow the synthetic sets, with a mistake sentence placed on a new line after a real message without closing punctuation so that it forms a sentence of its own.

F Timeline of registrations and measurements

Table 11: Commits (UTC, 19 October 2026) that registered each section, triggered its measurement and added its results. Test users were committed before any measurement on them (§8: 234bab0; §9: d405a03; §10: 9831eed; real users: 41431d1; §12 users and probes: 789abea); the §12 cue words were frozen in 96e9897 (18:34).

Section	Registered	Measurement triggered	Results committed
§7	a594d98 (00:28), amended §7.1–7.4 until 04:36	296c62d (04:36)	b12fe09 (04:41)
§8	b59414a (10:20)	bbfa8e0 (10:21)	15e59bc (10:29)
§9	bed30a6 (13:02)	e0685aa, 2917afc (13:02)	88f7f30 (13:10), b41a222 (13:18)
§10	87a4c4d (13:22)	ca923a0 (13:22), 26c5924 (13:23)	203881f (13:30), 039d25c (13:38)
§11	3fb881a (18:10; same commit)	3fb881a	975d994 (18:19)
§12	789abea (18:38; same commit)	789abea, e2f3790 (18:38)	8b4e2fc (18:48), 106956f (18:56)

G Discrepancies between documents and result files

We report the result files wherever the project's prose documents differ from them. (i) The preregistration's tables for §9.1 and §10.1 gave the off-arm L4 count as 2 and 3; the result files record 1 (§9) and 2 (§10), with limits of 3 and 4. No decision changes; both sections now carry a dated correction note. (ii) The model card attributed 10 stored mistakes to §10; the result files record 10 for §8 and 5 for §10. The model card has been corrected. (iii) Counts of replies that printed the note itself in §8 cannot be reproduced exactly with the pattern in the current code, so we do not report them.